LAB 02: Python Data Structures - List · Dict · NumPy · Pandas

Duration: 3 Hours | Course: Introduction to Machine Learning | Spring 2026

Lab Objectives

- Perform all CRUD operations on Python lists using built-in methods
- Write list comprehensions for filtering, mapping, and flattening
- Build, access, and iterate Python dictionaries and nested dicts
- Use dict comprehensions and handle missing keys safely
- Create and manipulate NumPy arrays (zeros, ones, arange, randn)
- Apply vectorised math, boolean indexing, and matrix operations
- Create, inspect, filter, and transform Pandas DataFrames
- Handle missing values and perform groupby aggregations

Lab Schedule (3 Hours)

TIME	PHASE	TOPICS
0:00–0:15	**Setup & Recap**	Install, quick Lab01 review
0:15–0:55	**List**	Creation, slicing, comprehension, tasks
0:55–1:35	**Dictionary**	CRUD, nesting, comprehension, tasks
1:35–1:50	**Break**	Rest / Q&A
1:50–2:25	**NumPy**	Arrays, shapes, math, stats
2:25–2:50	**Pandas**	DataFrame, filter, groupby, fill NaN
2:50–3:00	**Wrap-up**	Take-home task, evaluation

Setup - Install & Verify

```
pip install numpy pandas openpyxl

# Verify in Python REPL or script
import numpy as np
import pandas as pd
print(np.version__)    # >= 1.24
print(pd.version__)    # >= 2.0
```

When to Use What

STRUCTURE	ORDERED?	MUTABLE?	KEY ACCESS?	BEST FOR IN ML
`list`	☑ Yes	☑ Yes	Index	Sequences, feature vectors, batches

`dict`	☑ Yes	☑ Yes	Named keys	Config, label maps, JSON/API data
`numpy`	☑ Yes	☑ Yes	Index	Matrix ops, model weights, math
`pandas`	☑ Yes	☑ Yes	Column names	Tabular data, EDA, preprocessing

Part 1 - Python List (0:15–0:55)

1.1 Creation, Access, Modification

```
# Creating lists
scores  = [85, 72, 90, 65, 88, 77]
labels  = ['cat', 'dog', 'bird']
nested  = [[1,2], [3,4], [5,6]]      # 2-D data

# Indexing & Slicing
print(scores[0])        # 85   (first element)
print(scores[-1])       # 77   (last element)
print(scores[1:4])      # [72, 90, 65]
print(scores[::2])      # every 2nd  [85, 90, 88]
print(scores[::-1])     # reversed

# Mutating
scores.append(95)       # add to end
scores.insert(2, 99)    # insert at index 2
scores.remove(65)       # remove first 65
print(scores.pop())     # remove & return last
scores.sort()           # in-place ascending sort
print(sorted(scores, reverse=True))  # new sorted copy
```

1.2 List Comprehension — Core ML Pattern

> *ML Connection: List comprehensions appear in every ML codebase — normalisation, filtering samples, building label arrays.*

```
# Syntax:  [expression  for  item  in  iterable  if  condition]

raw = [10, 20, 30, 40, 50]

# 1. Transform every element
doubled    = [x * 2 for x in raw]

# 2. Normalize to [0, 1]  ← used in every ML pipeline
normalized = [x / max(raw) for x in raw]
print(normalized)   # [0.2, 0.4, 0.6, 0.8, 1.0]

# 3. Filter elements
above_25   = [x for x in raw if x > 25]

# 4. Transform + filter together
scaled     = [x**2 for x in raw if x % 20 == 0]

# 5. Flatten a 2-D list
matrix     = [[1,2],[3,4],[5,6]]
```

```
flat       = [val for row in matrix for val in row]
print(flat)         # [1, 2, 3, 4, 5, 6]
```

Task 1 - List Operations (10 min, individual)

Given a list of exam scores, write ONE line of code for each sub-task:

```
scores = [45, 82, 56, 91, 38, 74, 88, 60]

# 1. All scores above 60
passed   = [s for s in scores if s > 60]

# 2. Normalize to 0–100 range using min-max formula
norm     = [(s - min(scores)) / (max(scores) - min(scores)) * 100
            for s in scores]

# 3. Count how many failed (< 60)
fail_cnt = len([s for s in scores if s < 60])

print(passed, fail_cnt)
```

Part 2 - Dictionary (0:55–1:35)

2.1 CRUD & Core Operations

```
# Create
student = {
    "name"  : "Alice",
    "id"    : 2210,
    "scores": [85, 90, 78],
    "active": True
}

# Read
print(student["name"])               # Alice
print(student.get("gpa", "N/A"))    # safe read with default

# Update
student["gpa"]  = 3.8              # add new key
student["name"] = "Alice Rahman"   # update existing

# Delete
del student["active"]
student.pop("id", None)             # safe pop (no KeyError)

# Inspection
print(student.keys())
print(student.values())
print(student.items())      # (key, value) tuples
"gpa" in student            # True
```

2.2 Loops, Comprehension & Nesting

```
grades = {"Alice": 92, "Bob": 74, "Clara": 88, "Daud": 61}
```

```
# Iterate with .items()
for name, score in grades.items():
    status = "Pass" if score >= 70 else "Fail"
    print(f"{name}: {status}")

# Dict comprehension
pass_fail = {k: ("Pass" if v >= 70 else "Fail") for k, v in grades.items()}

# Word frequency counter (NLP use-case)
text = "the quick brown fox jumps over the lazy dog the fox"
freq = {}
for word in text.split():
    freq[word] = freq.get(word, 0) + 1

# Nested dict — ML model config
model_config = {
    "model" : "RandomForest",
    "params": {"n_estimators": 100, "max_depth": 5},
    "metrics": {"accuracy": 0.91, "f1": 0.89}
}
print(model_config["params"]["max_depth"])  # 5

# dict.update() — merging configs
overrides = {"max_depth": 10, "min_samples": 2}
model_config["params"].update(overrides)
```

Task 2 - Gradebook Dictionary (10 min, pair work)

Build a class gradebook as a nested dictionary, then print only passing students.

```
gradebook = {}
entries   = [("Alice",85), ("Bob",72), ("Clara",90), ("Daud",58)]

for name, score in entries:
    grade = "A" if score >= 80 else "B" if score >= 70 else "F"
    gradebook[name] = {"score": score, "grade": grade}

# Print only students who passed
for name, info in gradebook.items():
    if info["grade"] != "F":
        print(f"{name}: {info['score']} → {info['grade']}")
```

Part 3 - NumPy (1:50–2:25)

ML Connection: NumPy arrays are the foundation of ALL ML frameworks. PyTorch tensors, TensorFlow tensors, and scikit-learn inputs are all built on ndarray. Knowing shapes = knowing ML.

3.1 Array Creation & Shapes

```
import numpy as np

# Creating arrays
a = np.array([1, 2, 3, 4, 5])
```

```
b = np.zeros((3, 4))           # 3x4 all zeros
c = np.ones((2, 3))            # 2x3 all ones
d = np.eye(3)                  # 3x3 identity matrix
e = np.arange(0, 10, 2)        # [0, 2, 4, 6, 8]
f = np.linspace(0, 1, 5)       # 5 evenly spaced in [0,1]
g = np.random.randn(4, 3)     # 4x3 standard normal

# Shape operations
m = np.array([[1,2,3],[4,5,6]])
print(m.shape)           # (2, 3)  ← rows x cols
print(m.ndim)            # 2
print(m.size)            # 6  (total elements)
print(m.reshape(3, 2))   # reshape without copy
print(m.flatten())        # → 1-D [1,2,3,4,5,6]
print(m.T)               # transpose → shape (3, 2)
```

3.2 Vectorised Math, Boolean Indexing & Stats

```
x = np.array([2, 4, 6, 8, 10])
y = np.array([1, 3, 5, 7, 9])

# Element-wise — NO loops needed!
print(x + y)          # [ 3  7 11 15 19]
print(x * y)          # [ 2 12 30 56 90]
print(x ** 2)         # [  4  16  36  64 100]
print(np.sqrt(x))     # square root element-wise

# Broadcasting — scalar applied to whole array
print(x + 100)        # [102 104 106 108 110]
print(x / x.max())    # normalize to [0,1]

# Boolean indexing
print(x[x > 5])       # array([ 6,  8, 10])
x[x < 5] = 0          # set values conditionally

# Statistics
data = np.random.randn(100)
print(f'Mean  : {np.mean(data):.4f}')
print(f'Std   : {np.std(data):.4f}')
print(f'Min   : {np.min(data):.4f}')
print(f'Max   : {np.max(data):.4f}')
print(f'Median: {np.median(data):.4f}')

# Matrix multiplication (dot product)
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])
print(A @ B)          # or np.dot(A, B)
# Note: A * B is element-wise, NOT matrix multiply!
```

Part 4 - Pandas (2:25–2:50)

ML Connection: 80% of ML work is data preparation. Pandas IS that tool. Every Kaggle notebook starts with: import pandas as pd

4.1 DataFrame Creation & Inspection

```
import pandas as pd

# Create from dictionary (most common way)
df = pd.DataFrame({
    "name"  : ["Alice", "Bob", "Clara", "Daud", "Eve"],
    "age"   : [22, 24, 21, 23, 22],
    "score" : [85, 72, 90, None, 88],    # None = missing!
    "dept"  : ["CSE", "CSE", "EEE", "CSE", "EEE"]
})

# Inspection
print(df)
print(df.shape)          # (5, 4)
print(df.head(3))        # first 3 rows
print(df.tail(2))        # last 2 rows
print(df.info())         # dtypes + null counts
print(df.describe())     # mean, std, min, max per column
print(df.isnull().sum()) # count NaN per column
```

4.2 Select, Filter, Add, Fill, GroupBy

```
# Selecting columns
print(df["name"])               # Series (single col)
print(df[["name", "score"]])    # DataFrame (multi col)

# Selecting rows
print(df.iloc[0])              # by integer position
print(df.iloc[1:3])            # slice of rows
print(df.loc[df["dept"] == "CSE"])  # label-based filter

# Filtering
high   = df[df["score"] > 80]
cse_hi = df[(df["dept"] == "CSE") & (df["score"] > 80)]

# Adding a computed column
df["grade"] = df["score"].apply(
    lambda s: "A" if s >= 85 else "B" if s >= 70 else "F"
)

# Handling missing data
df["score"].fillna(df["score"].mean(), inplace=True)  # fill NaN with mean
# df.dropna(inplace=True)   # or drop rows with any NaN

# GroupBy — average score per department
print(df.groupby("dept")["score"].mean())

# Sort
print(df.sort_values("score", ascending=False))

# Save to CSV
df.to_csv("output.csv", index=False)
```

Take-Home Assignment - Mini EDA Pipeline

Due: Start of next lab session. Submit your .py file and the generated output files.

```
# Dataset: Create a file 'students.csv' with columns:
#   name, department, math_score, english_score, science_score
#   At least 10 students, at least 2 departments.
#   Include at least 2 missing values somewhere.

# Required tasks:
# 1. Load the CSV with Pandas
# 2. Show shape, dtypes, and null counts (.info())
# 3. Fill missing scores with the column mean
# 4. Add column 'avg_score' = mean of the 3 score columns per student
# 5. Use a DICT COMPREHENSION to build: {name → 'Pass'/'Fail'}
# 6. Convert avg_score column to NumPy array;
#    compute and print mean, std, min, max
# 7. Print a sorted leaderboard (name + avg_score), highest first

# BONUS (+5): Group by department, print average avg_score per dept
```

Quick Reference - Cheat Sheet

List create	`lst = [1, 2, 3]  list(range(5))`
Slice	`lst[1:4]  lst[::-1]  lst[::2]`
Mutate	`append()  insert()  remove()  pop()  sort()`
Comprehension	`[x*2 for x in lst if x > 0]`
Dict create	`d = {'key': val}  dict(a=1, b=2)`
Read	`d['key']  d.get('key', default)`
Loop	`for k, v in d.items():`
Dict comp.	`{k: v*2 for k, v in d.items()}`
Array create	`np.array([])  np.zeros()  np.ones()  np.arange()`
Shape	`.shape  .ndim  .size  .reshape()  .flatten()  .T`
Math	`np.mean()  std()  min()  max()  dot()  sqrt()`
Bool index	`arr[arr > 5]  arr[arr < 0] = 0`
Create	`pd.DataFrame({'col': [...]})  pd.read_csv('f.csv')`
Inspect	`.shape  .info()  .describe()  .head()  .isnull().sum()`
Select	`df['col']  df[['a','b']]  df.iloc[0]  df.loc[mask]`
Filter	`df[df['col'] > 5]  df[(cond1) & (cond2)]`
Transform	`df['new'] = df['col'].apply(fn)`
Missing	`fillna(val)  dropna()  .isnull()`
Groupby	`df.groupby('dept')['score'].mean()`